

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24

Representing the SKI Combinator Calculus in the Nock Instruction Set Architecture

N. E. Davis ~lagrev-nocfep
ZeroAttest, Groundwire Foundation

Abstract

The Nock ISA opcodes 2, 1, and 0 have been said to correspond to the combinators S, K, and I in the SKI combinator calculus. We give a four-rule compiler from arbitrary SKI terms to Nock formulas using only opcodes 0, 1, and 2, prove it correct under a fixed application convention, and exercise it on the usual worked examples: the identity laws, Church booleans, the B, C, and W combinators, and Church numerals performing real arithmetic. The compiled output is verified against an independent Nock 4K interpreter. The construction is a constructive proof that the Nock fragment of 0, 1, 2 opcodes is Turing-complete. This fragment lacks certain properties, marking a boundary beyond which the fragment is genuinely insufficient and additional tools are required.

Contents

1	Introduction	2
2	Preliminaries	4

25	3	Combinator Formulas	5
26	3.1	I identity	5
27	3.2	K constant	5
28	3.3	S substitution	6
29	3.4	B composition	6
30	3.5	C transposition	7
31	3.6	W duplication	7
32	3.7	Commentary	7
33	4	Compilation of the Homomorphism	8
34	5	Worked Examples	9
35	5.1	Identity laws	10
36	5.2	Church booleans	12
37	5.3	The B, C, W combinators	13
38	5.4	Church numerals and arithmetic	14
39	6	Validation	17
40	7	Cost	17
41	8	Limitations	18
42	9	Conditionals	19
43	10	Conclusion	20

1 Introduction

The Nock instruction set architecture is a combinator calculus sufficient to represent all computable functions. It has been a longstanding temptation to draw a direct analogy from the Nock ISA to the SKI combinator calculus. This analogy claims that the Nock opcodes 2, 1, and 0 correspond to the combinators S, K, and I respectively. The analogy is usually stated informally, e.g. “[there are] some subtle differences to Nock’s expression of S as opcode 2 that we will elide as being fundamentally similar, but perhaps worthy of its own monograph” (~lagrev-nocfep and ~sorreg-namtyv, 2025). We here deliver

55 upon that promise and convert an Urbit legend into a concrete
56 demonstration.

57 Combinator calculi build all computation from a handful
58 of variable-free operators; the SKI system does it with three
59 (Curry and Feys, 1958), which is neither unique nor minimal
60 but balances convenience with economy.¹ Put briefly, the S
61 combinator (“substitute”) takes three arguments and applies
62 the first to the third, then applies the second to the third, and
63 finally applies the result of the first application to the result
64 of the second. The K combinator (“k’onstant”) takes two ar-
65 guments and returns the first, discarding the second. The I
66 combinator (“identity”) takes one argument and returns it un-
67 changed. The SKI system is Turing-complete, meaning that
68 any computable function can be expressed as a combination
69 of these three operators.

70 $S\ x\ y\ z = x\ z\ (y\ z)$

71 $K\ x\ y = x$

72 $I\ x = x$

73 The Nock ISA is also Turing-complete, and it is natural to
74 ask whether the SKI combinators can be represented in Nock.
75 The answer is yes, but some details are subtle. The analogy
76 between Nock opcodes and SKI combinators is not a simple
77 one-to-one compilation procedure, but a homomorphism that
78 preserves the structure of the computation.

79 A straightforward reading of Nock opcode 0 yielding I is
80 uncontroversial, as [0 1] trivially returns identity. The read-
81 ing of opcode 1 yielding K is also straightforward, as [1 x]
82 returns a constant function that discards its argument (sub-
83 ject). The reading of opcode 2 yielding S is more subtle, as it
84 requires a fixed convention for how the compiled combinator
85 receives its arguments. The “subtle differences” of *~lagrev-*
86 *nocfep* and *~sorreg-namtyv*’s elision matter more than they
87 appear. The opcode-to-combinator analogy is not a compila-
88 tion procedure. SKI terms are curried and higher-order: com-
89 binators are routinely partially applied, and a combinator may
90 be passed as an argument to another combinator. A symbol-
91 for-symbol substitution—writing 2 for S, 1 for K, and 0 for I,

¹See particularly Smullyan (1994) for more on combinatory logic.

with appropriate transposition of arguments into subject and formula—produces nothing runnable the moment S has fewer than three arguments, which is almost always. What is needed is a *homomorphism*: a translation under which Nock application of compiled terms mirrors SKI reduction, with a fixed convention for how a compiled combinator receives its arguments.

We supply such a homomorphism as four rules with three small Nock formulas yielding the SKI combinators. The remainder of this article consists of worked examples, mechanical validation, a cost analysis, and an accounting of the limitations.

2 Preliminaries

We assume Nock 4K. Evaluation is the function $\star[\text{subject formula}]$. We require only three opcodes from the Nock ISA:

```

:: slot: fetch the noun at tree address b
 $\star[a\ 0\ b] \rightarrow /[b\ a]$ 

:: quote: return b unchanged
 $\star[a\ 1\ b] \rightarrow b$ 

:: eval: compute a subject and a formula, then run
 $\star[a\ 2\ b\ c] \rightarrow \star[\star[a\ b]\ \star[a\ c]]$ 

```

plus the structural distribution (autocons) rule, which fires whenever a formula's head is itself a cell:

```

 $\star[a\ [b\ c]\ d] \rightarrow [\star[a\ b\ c]\ \star[a\ d]]$ 

```

This distribution rule is not an opcode but the structural behavior of $\star$ when a formula's head is a cell; it is Nock's only means to construct a cell from evaluated components. Basically every combinator formula in this paper has a cell head and so depends on it; the construction therefore uses opcodes 0, 1, 2 together with distribution.

Slot address 1 denotes the whole subject, so $\star[a\ 0\ 1] = a$.

A *value* is a Nock formula that consumes its argument as the *subject*. To apply a value v to an argument x , we evaluate

127 `apply(v, x) == *[x v]`

128 This operative convention inverts the lambda-calculus argu-
 129 ment order, because in Nock the function is the formula and
 130 the argument is the subject.

131 Prior art exists for SKI using the Nock ISA carried out by
 132 `~hex` (2024) in Racket and miniKanren; his strategy for S and I
 133 anticipated and are identical to ours, but his formulation of K
 134 relied on Nock opcode 8, as well as opcodes 3 and 6 for adap-
 135 tively partial application, which is out of scope for our argu-
 136 ment. (We resolve the tension of partial application via au-
 137 tocons rather than opcode 8.) `~hex` permitted the outer entry
 138 point `nock(E)` to accept an expression E written in a “Nock In-
 139 termediate Representation” syntax, that is, with `*` annotations
 140 in front of sub-expressions to force an evaluation order. The
 141 Racket code also has a second, concatenative encoding (`I = [0`
 142 `1]`, `K = [0 3]`, `S = [[0 2] [[1 2] [0 3]]]`) for which all argu-
 143 ments sit in the subject as a stack. This is elegant and concise
 144 but, as he notes, it cannot curry.² Finally, `~hex` also introduced
 145 a formulation of the Z combinator as the strict fixed point for
 146 eagerly evaluated languages.

147 3 Combinator Formulas

148 Under the convention of Section 2, each SKI combinator is a
 149 closed Nock formula. We give each, then verify its defining
 150 law by reduction.

151 3.1 I identity

152 `I x = x` requires `*[x I] = x`, which is slot 1:

153 `I = [0 1]`

154 3.2 K constant

155 Applying K to x must yield a value that ignores its next argu-
 156 ment and returns its value. Thus `K x y = x` requires `*[x K]`

²This approach merits further investigation.

157 = [1 x], which is built from subject x by autocons:

158 $K = [[1\ 1]\ [0\ 1]]$

159 For example,

160 $*[x\ K] = [*[x\ [1\ 1]]\ *[x\ [0\ 1]]] = [1\ x]$

161 Then for any y, $*[y\ [1\ x]] = x$, so $K\ x\ y = x$ holds.

162 3.3 S substitution

163 Substitution states $S\ x\ y\ z = (x\ z)(y\ z)$. The combinator
164 accumulates x, then y, then fires on z. We construct it bottom-
165 up.

166 When the fully-applied $S\ x\ y$ finally receives z as subject,
167 it must produce $(x\ z)(y\ z)$. Under the convention, $x\ z =$
168 $*[z\ x]$ and $y\ z = *[z\ y]$, and the outer application is $*[$
169 $*[z\ y]\ *[z\ x]]$. That is precisely opcode z with subject z,
170 formula-b equal to y, formula-c equal to x:

171 $S\ x\ y = [2\ y\ x]$

172 $::\ *[z\ [2\ y\ x]] = [*[z\ y]\ *[z\ x]] = (x\ z)(y\ z)$

173 Working back one step, applying $S\ x$ to y (subject y, constant
174 x) must build the noun $[2\ y\ x]$:

175 $S\ x = [[1\ 2]\ [0\ 1]\ [1\ x]]$

176 We check that $*[y\ S\ x] = [2\ y\ x]$, since $*[y\ [1\ 2]] =$
177 2 , $*[y\ [0\ 1]] = y$, and $*[y\ [1\ x]] = x$. Working back the
178 final step, applying S to x (subject x) must build $S\ x$; the first
179 two parts are constants and the third, $[1\ x]$, is built from x by
180 the same autocons trick used for K:

181 $S = [[1\ [1\ 2]]\ [1\ [0\ 1]]\ [[1\ 1]\ [0\ 1]]]$

182 We check the evaluation: $*[x\ S] = [[1\ 2]\ [0\ 1]\ [1\ x]]$
183 $= S\ x$.

184 3.4 B composition

185 Any other combinator can be reached by elaborating its lambda
186 definition to SKI and compiling, but the result is large. A named
187 combinator is better derived directly, by the same method as

the previous subsection: it becomes one builder layer per argument. The innermost layer is the opcode 2 application skeleton that fires on the final argument; each enclosing layer is an autocons that captures one argument and embeds it. We here derive B, C, and W; a similar methodology could be used for much of Smullyan's aviary (Smullyan, 1994).

For $B \ f \ g \ x = f \ (g \ x)$, the firing step computes f applied to $g \ x$, i.e. $*[*[x \ g] \ f]$, which is opcode 2 with g in the formula-b slot and f quoted in the formula-c slot; capturing g then f gives:

```

198 :: * [x B f g] = * [* [x g] f] = f (g x)
199 B f g = [2 g [1 f]]
200 :: * [g B f] = [2 g [1 f]]
201 B f = [[1 2] [0 1] [1 [1 f]]]
202 ::
203 B = [[1 [1 2]] [1 [0 1]] [[1 1] [[1 1] [0 1]]]]

```

204 3.5 C transposition

205 The same procedure yields C (flip):

```

206 :: f x y -> f y x
207 C = [[1 1 2] [1 [1 1] 0 1] [1 1] 0 1]

```

208 3.6 W duplication

209 Likewise, W (duplicate):

```

210 :: f x -> f x x
211 W = [[1 2] [1 0 1] 0 1]

```

212 3.7 Commentary

213 The one subtlety is how a captured argument is embedded,
 214 which is dictated by how the combinator uses that argument. If
 215 the argument is applied to a computed value (as B's f is applied
 216 to $g \ x$), it occupies a formula slot and is embedded quoted, $[1$
 217 $f]$. If it is applied to the current subject (as W's f is applied to
 218 x) it must pass through as the live formula, embedded by slot,

Table 1: Compiled Nock cell counts for various SKI terms, using direct derivation rather than bracket abstraction; compare to Table 3.

Combinator	Direct	Via SKI	Ratio
B	11	208	$\approx 19\times$
C	11	208	$\approx 19\times$
W	6	130	$\approx 22\times$

[0 1]. Quote where the argument is data; slot where the argument is the active function.

Notably, the value forms of B, C, and W use only Nock opcodes 0 and 1. The opcode 2 appears only in the formula they assemble at application time ([2 g [1 f]]), carried as quoted data. The combinators are pure construction; the application machinery is the structure they emit. Among the basis only S carries a 2 in its own body, because it performs two applications at once.

The size payoff over bracket abstraction is large (cell counts; see Section 7 for method), as recorded in Table 1.

The two forms are extensionally identical on every input tested. For any combinator known by name, direct derivation is preferred and the abstraction tax is avoided entirely; bracket abstraction remains the general fallback for arbitrary lambda terms.

4 Compilation of the Homomorphism

Let [[t]] denote a *closed* Nock formula whose product is the value of the SKI term t. The compiler consists of four rules from bracket abstraction to Nock ISA:

[[S]] = [1 S] = [1 [1 [1 2]] [1 [0 1]] [[1 1] [0 1]]]
 [[K]] = [1 K] = [1 [1 1] [0 1]]
 [[I]] = [1 I] = [1 0 1]
 [[A B]] = [2 [[B]] [[A]]]

with S, K, I the formulas of Section 3. The combinators are *quoted* so that [[·]] always denotes a value-producing com-

putation; application is opcode 2 applied to the two compiled operands. $[[t]]$ produces the value of t ; it is closed, so the naïve approach of $*[a \ [[t]]]$ yields $\text{val}(t)$ for any subject a and is not yet an application. To apply t to Nock-noun arguments $a[0] \ \dots \ a[n]$, fold them onto the value, each quoted, by opcode 2:

```
run( $t$ ;  $a[1] \ \dots \ a[n]$ ) =  $*[0 \ F[n]]$ 
```

where $F[0] = [[t]]$, $F[i] = [2 \ [1 \ a[i]] \ F[i-1]]$. Each layer $*[\cdot \ [2 \ [1 \ a[i]] \ F]]$ reduces to $*[a[i] \ \text{val}]$, the value applied to $a[i]$ as subject. Throughout the remainder of this paper, we will write $t \ a[1] \ \dots \ a[n] = r$ as shorthand for $\text{run}(t; a[1] \ \dots \ a[n]) = r$. The arguments are Nock nouns, not SKI terms: atoms in the identity and boolean laws, real formulas such as $+/INC$ in the arithmetic examples. A tap must not fire until its formula is a concrete noun.

Why quote, and why this order? Evaluating $[[A \ B]] = [2 \ [[B]] \ [[A]]]$ against any subject s gives $*[*[s \ [[B]]] \ *[*[s \ [[A]]]]]$. Because $[[B]]$ and $[[A]]$ are closed, they ignore s and reduce to the values $\text{val}(B)$ and $\text{val}(A)$. The result is $*[\text{val}(B) \ \text{val}(A)]$ —apply value $\text{val}(A)$ to argument $\text{val}(B)$, i.e. A applied to B . The operands are reduced to values *before* application, which is what makes nested applications compose correctly; quoting a sub-application rather than evaluating it produces an “off-by-one error” (a residual K where the value was expected).

Correctness is demonstrated practically by induction on term structure. For application, the rule computes $\text{apply}(\text{val}(A), \text{val}(B))$, and by the induction hypothesis $\text{val}(A), \text{val}(B)$ are the values of A, B ; by §2 this is the value of $A \ B$. The translation is therefore a homomorphism from SKI application to Nock evaluation, under the call-by-value strategy forced by opcode 2’s eager semantics (see Section 8).

277 5 Worked Examples

278 All terms below are written in lambda form for legibility, elab-
279 orated to SKI by standard bracket abstraction (Kiselyov, 2018),

280 then compiled by the Nock SKI compiler and run. The results
281 are the products actually computed.

282 A script which resolves the SKI terms to Nock formulas,
283 runs them in a Nock 4K interpreter, and checks the results is
284 available at [sigilante/skitrace](#).

285 5.1 Identity laws

286 From canonical SKI, we anticipate the following identity laws
287 to hold true:

```
288 I 42      = 42
289 S K K 42  = 42      :: S K K → I
290 S K S 42  = 42      :: S K * → I
```

291 The evaluation of $S\ K\ K\ 42$, worked in Nock SKI, is shown in
292 Listings 1, 2, and 3. The process is mechanical and discursive.
293 The evaluation of $S\ K\ S\ 42$ is similar and left as a proverbial
294 exercise to the reader.

Listing 1: Compilation expansion of $S\ K\ K\ 42$.

```
295 *[42 [[S K K]]]
296 *[42 2 [[K]] [[S K]]]
297               [[A B]] = [2 [[B]] [[A]]]
298 *[42 2 [1 K] [[S K]]]
299               [[K]] = [1 K]
300 *[42 2 [1 K] 2 [[K]] [[S]]]
301               [[A B]] = [2 [[B]] [[A]]]
302 *[42 2 [1 K] 2 [1 K] [[S]]]
303               [[K]] = [1 K]
304 *[42 2 [1 K] 2 [1 K] 1 S]
305               [[S]] = [1 S]
306 *[42 2 [1 [1 1] 0 1] 2 [1 [1 1] 0 1] 1 [1 1 2] [1 0 1] [1
307   1] 0 1]
308               substitute S,K,I formulas
```

Listing 2: Evaluation of $S\ K\ K\ 42$.

```
309 *[42 2 [1 [1 1] 0 1] 2 [1 [1 1] 0 1] 1 [1 1 2] [1 0 1] [1
310   1] 0 1]
311               Nock 2  *[a 2 b c] = *[*[a b] *[a c]]
312 *[*[42 1 [1 1] 0 1] *[42 2 [1 [1 1] 0 1] 1 [1 1 2] [1 0
313   1] [1 1] 0 1]]
```

Representing SKI in Nock ISA

```

314           Nock 1  *[a 1 b] = b
315  *[[[1 1] 0 1] *[42 2 [1 [1 1] 0 1] 1 [1 1 2] [1 0 1] [1
316    1] 0 1]]
317           Nock 2  *[a 2 b c] = *[*[a b] *[a c]]
318  *[[[1 1] 0 1] *[*[42 1 [1 1] 0 1] *[42 1 [1 1 2] [1 0 1]
319    [1 1] 0 1]]]
320           Nock 1  *[a 1 b] = b
321  *[[[1 1] 0 1] *[[[1 1] 0 1] *[42 1 [1 1 2] [1 0 1] [1 1]
322    0 1]]]
323           Nock 1  *[a 1 b] = b
324  *[[[1 1] 0 1] *[[[1 1] 0 1] [1 1 2] [1 0 1] [1 1] 0 1]]
325    cons  *[a [b c] d] = *[*[a b c] *[a d]]
326  *[[[1 1] 0 1] *[[[1 1] 0 1] 1 1 2] *[[[1 1] 0 1] [1 0 1]
327    [1 1] 0 1]]
328           Nock 1  *[a 1 b] = b
329  *[[[1 1] 0 1] [1 2] *[[[1 1] 0 1] [1 0 1] [1 1] 0 1]]
330    cons  *[a [b c] d] = *[*[a b c] *[a d]]
331  *[[[1 1] 0 1] [1 2] *[[[1 1] 0 1] 1 0 1] *[[[1 1] 0 1]
332    [1 1] 0 1]]
333           Nock 1  *[a 1 b] = b
334  *[[[1 1] 0 1] [1 2] [0 1] *[[[1 1] 0 1] [1 1] 0 1]]
335    cons  *[a [b c] d] = *[*[a b c] *[a d]]
336  *[[[1 1] 0 1] [1 2] [0 1] *[[[1 1] 0 1] 1 1] *[[[1 1] 0
337    1] 0 1]]
338           Nock 1  *[a 1 b] = b
339  *[[[1 1] 0 1] [1 2] [0 1] 1 *[[[1 1] 0 1] 0 1]]
340           Nock 0  *[a 0 b] = /[b a]
341  *[[[1 1] 0 1] [1 2] [0 1] 1 /[1 [[1 1] 0 1]]]
342    /[1 a] = a
343  *[[[1 1] 0 1] [1 2] [0 1] 1 [1 1] 0 1]
344    cons  *[a [b c] d] = *[*[a b c] *[a d]]
345  *[[[1 1] 0 1] 1 2] *[[[1 1] 0 1] [0 1] 1 [1 1] 0 1]]
346           Nock 1  *[a 1 b] = b
347  [2 *[[[1 1] 0 1] [0 1] 1 [1 1] 0 1]]
348    cons  *[a [b c] d] = *[*[a b c] *[a d]]
349  [2 *[[[1 1] 0 1] 0 1] *[[[1 1] 0 1] 1 [1 1] 0 1]]
350           Nock 0  *[a 0 b] = /[b a]
351  [2 /[1 [[1 1] 0 1]] *[[[1 1] 0 1] 1 [1 1] 0 1]]
352    /[1 a] = a
353  [2 [[1 1] 0 1] *[[[1 1] 0 1] 1 [1 1] 0 1]]
354           Nock 1  *[a 1 b] = b
355  [2 [[1 1] 0 1] [1 1] 0 1]

```

356 (= [2 K K], the identity combinator's value)

Listing 3: Application of S K K 42.

```

357 # applying val(S K K) to 42  ( *[42 val(S K K)] )
358 val(S K K) = [2 [[1 1] 0 1] [1 1] 0 1]
359
360 # evaluation of *[42 [2 [[1 1] 0 1] [1 1] 0 1]]
361 *[42 2 [[1 1] 0 1] [1 1] 0 1]
362       Nock 2  *[a 2 b c] = *[*[a b] *[a c]]
363 *[*[42 [1 1] 0 1] *[42 [1 1] 0 1]]
364       cons  *[a [b c] d] = *[a b c] *[a d]]
365 *[[*[42 1 1] *[42 0 1]] *[42 [1 1] 0 1]]
366       Nock 1  *[a 1 b] = b
367 *[[1 *[42 0 1]] *[42 [1 1] 0 1]]
368       Nock 0  *[a 0 b] = /[b a]
369 *[[1 /[1 42]] *[42 [1 1] 0 1]]
370       /[1 a] = a
371 *[[1 42] *[42 [1 1] 0 1]]
372       cons  *[a [b c] d] = *[a b c] *[a d]]
373 *[[1 42] *[42 1 1] *[42 0 1]]
374       Nock 1  *[a 1 b] = b
375 *[[1 42] 1 *[42 0 1]]
376       Nock 0  *[a 0 b] = /[b a]
377 *[[1 42] 1 /[1 42]]
378       /[1 a] = a
379 *[[1 42] 1 42]
380       Nock 1  *[a 1 b] = b
381 42

```

5.2 Church booleans

Conventional Nock utilizes atoms, but SKI combinators are pure construction; we therefore must utilize Church-style booleans and numerals within the particular three-opcode discipline we follow here. Thus while this section is demonstrative of SKI in Nock, it evades the usual Nock idioms and in particular the economy afforded by the introduction of opcodes 3, 4, and 5.

A Church boolean is a two-argument function that selects one of its arguments. Because the combinators are pure con-

struction, the truth values are instead represented as combinator sequences. With `TRUE = K` and `FALSE = K I`:

```

392 TRUE p q
393   = K p q
394   = p
395
396 FALSE p q
397   = K I p q
398   = I q
399   = q
400
401 TRUE 7 8 = 7
402
403 FALSE 7 8 = 8
404
```

These are the canonical encodings from SKI praxis; here we probe with atom arguments because neither combinator applies its arguments as functions.

5.3 The B, C, W combinators

Elaborated from their defining lambda terms:

```

410 B = λ f g x. f (g x)    compose
411 C = λ f x y. f y x      transpose
412 W = λ f x. f x x        duplicate

```

Driving B with a real Nock increment `inc = [4 0 1]` as payload, and C, W with atoms:

```

415 B inc inc 7 = 9      :: inc (inc 7)
416 C K 7 8     = 8      :: K 8 7 = 8
417 W K 7       = 7      :: K 7 7 = 7

```

These exercise three-variable abstraction and argument duplication. Note that the compiled SKI scaffolding remains pure `o`, `1`, `2`; the only `4` in B's run is the external increment supplied as data.³

³We here draw a sharp distinction between opcode `4` as `inc = [4 0 1]` which operates on Nock atoms, and `succ` which operates on Church numerals. The two are distinct: `succ` maps a numeral-value to a numeral-value in `o,1,2`, while `inc` maps an atom to an atom via opcode `4`.

5.4 Church numerals and arithmetic

A Church numeral `church n` applies its first argument `n` times to its second. Numerals are built and incremented entirely within the fragment.⁴ Examples of Church numerals and their Nock SKI representations are given in Table 2. At this point, we will revert in part to the lambda calculus notation for clarity, but the underlying Nock SKI formulas are always available and are used in the actual evaluation. The successor function is defined by the usual combinator formula:

```
church n = λλf.x. f^n x
succ = λλλn.f.x. f (n f x)

succ (church 0) = church 1
succ (church 1) = church 2
```

In explicit Nock SKI form, the Church numeral successor function becomes:

```
succ = [[1 1 2] [0 1] 1 [1 1] 0 1]

church 2 = [[1 2] [[1 2] [1 0 1] [1 1] 0 1]
             [1 1] 0 1]
[church 2] = 2
succ (church 2) = [[1 2] [[1 2] [[1 2] [1 0 1] [1 1] 0 1]
                             [1 1] 0 1] [1 1] 0 1]
[succ (church 2)] = 3
```

Arithmetic over the Church numerals is defined by the usual combinator formulas:

```
:: in Church numerals
plus = λλλλm.n.f.x. m f (n f x)
mult = λλλm.n.f. m (n f)

:: alternatively
plus = λm n. m succ n
mult = λm n. m (plus n) (church 0)
```

⁴A Church numeral is a value (formula), not an atom. To read it out as a Nock atom we decode it: apply it to the atom successor `inc = [4 0 1]` and the atom 0, writing `[n]= n inc 0`: `[church 0]= 0` ; `[SUCC*5 (church 0)]= 5` ; `[SUCC (church 2)]= 3`. Decoding is the only step that uses opcode 4, and it is readout, not arithmetic.

Representing SKI in Nock ISA

```
455
456 :: compiled in Nock SKI form
457 plus = [[1 1 1 2] [1 0 1] [1 1 1 1] [1 1] 0 1]
458 mult = [[1 1 2] [1 0 1] [1 1 1] [1 1] 0 1]
459
460 :: in Nock decoding
461 [church 0] = 0
462 [church 3] = 3
463 [church 5] = 5
464 [plus 2 3] = 5
465 [mult 3 4] = 12
```

Table 2: Church numerals and their Nock SKI representations. The numeral `church n` is a value (formula) that applies its first argument `n` times to its second. The decoding `[n]= n inc 0` produces the Nock atom corresponding to the numeral.

n	[church n]	church n
0	0	[1 0 1]
1	1	[[1 2] [1 0 1] [1 1] 0 1]
2	2	[[[1 2] [[1 2] [1 0 1] [1 1] 0 1] [1 1] 0 1] [1 1] 0 1]
3	3	[[[1 2] [[1 2] [[1 2] [1 0 1] [1 1] 0 1] [1 1] 0 1] [1 1] 0 1] [1 1] 0 1]
4	4	[[[1 2] [[1 2] [[1 2] [[1 2] [1 0 1] [1 1] 0 1] [1 1] 0 1] [1 1] 0 1] [1 1] 0 1] [1 1] 0 1]
5	5	[[[1 2] [[1 2] [[1 2] [[1 2] [[1 2] [1 0 1] [1 1] 0 1] [1 1] 0 1] [1 1] 0 1] [1 1] 0 1] [1 1] 0 1]

Table 3: Compiled Nock cell counts for various SKI terms. These numbers are via bracket abstraction; compare to Table 1.

Term	SKI leaves	Nock cells
I	1	2
S K K	3	22
W	16	130
B	25	208
C	25	208
church 2	76	640
church 5	187	1588
church 10	372	3168

6 Validation

Any valid Nock ISA interpreter should correctly evaluate the examples above after the SKI compiler stage has been applied. We utilized both a simple interpreter with only opcodes 0, 1, and 2,⁵ and the `pinochle` interpreter with the full Nock ISA.⁶ Results were identical (and correct) in both cases. The examples are not exhaustive, but they exercise the combinators and the compiler in a variety of ways, including nested applications, argument duplication, and Church-style numerals and booleans.

7 Cost

The transformation is faithful but not frugal. Measuring SKI leaf count against compiled Nock cell count shows the extreme verbosity of the latter. Table 3 shows the compiled size of several terms, including the Church numerals and the named combinators.

Compiled Nock runs roughly $8.5\times$ the SKI leaf count (a constant per combinator plus the opcode 2 application wrapper) and the SKI itself is already exponential in the worst case

⁵`sigilante/skitrace`

⁶`sigilante/pinochle`.

under naive bracket abstraction. The natural number ten compiles to a 3168-cell formula as a Church numeral. These figures are static, before reduction; at runtime S duplicates its argument into both branches with no sharing, so evaluation cost compounds on representation cost.

Named combinators escape this blow-up before the compiler stage: derived directly rather than routed through bracket abstraction, B , C , and W are roughly twenty times smaller. The cost above is the price of the *general* compiler on arbitrary terms, not a property of the targets themselves.

8 Limitations

Three properties bound the construction. Each is a real boundary, not a deficiency to be patched away within the fragment.

1. **Call-by-value.** Opcode 2 is eager: $\star[a \ 2 \ b \ c]$ fully reduces $\star[a \ b]$ and $\star[a \ c]$ before applying. The compilation is therefore applicative-order. A term whose normal form depends on discarding a divergent subterm will loop. Concretely, with $\mathbb{W} = S \ I \ I$ and $\Omega = \mathbb{W} \ \mathbb{W}$, normal-order $K \ I \ \Omega$ reduces to I , but the compiled form diverges. For a compilation target this is usually acceptable; for a faithful lazy surface it is not, and recovering normal order requires explicit thunking, which reintroduces runtime branching and pulls in opcodes 3 and 6.

2. **No readback in $\{0, 1, 2\}$.** Compiled SKI can be executed but its normal form cannot be printed without distinguishing a residual S from a K from an application at runtime. For Nock, this requires a structural test (opcode 3), an equality test (opcode 5), and a conditional (opcode 6). The fragment is a faithful execution target, not a normalizer.

A normalizer must additionally return the normal form in an inspectable form, which means recognizing combinators at runtime, which the $\{0, 1, 2\}$ fragment cannot do. In other words, when an SKI evaluator completes,

it has a normal form or a tree whose leaves are S, K, I and whose internal nodes are applications. To print that normal form (to “read it back”), one walks the tree and at each node decides if it is a leaf or an application, and if a leaf, which combinator?

3. **No sharing.** This is tree reduction. S copies its argument with no sharing, so terms with shared redexes blow up exponentially. This is moderately acceptable as an intermediate representation to compile through, but impractical as a runtime for large terms, which want graph reduction.

These can be read as an alternate set of motivations for opcodes beyond 2; as may be surmised, the {0, 1, 2} fragment is Turing-complete but lacks certain practical features.

9 Conditionals

A conditional is an obvious stress test for such a small fragment because in a strict evaluator a conditional that evaluates both branches is useless for its principal job of guarding recursion. In practice, the conditional must be split into two parts.

The Church booleans are already a form of conditional; with `TRUE = K` and `FALSE = K I`, the term `b t e` routes to the chosen branch. The routing itself executes neither branch: K reads its two arguments by slot and discards one, never applying them as formulas. If the branches are held as quoted formulas (thunks) and only the survivor is run, the unchosen branch is never evaluated. The whole conditional is one closed formula in the fragment:

```
IF cond tf ef s
    = [2 [1 s] [2 [1 ef] [2 [1 tf] [1 cond]]]]
```

Reading inner to outer: apply `cond` to `tf`, then to `ef` (selection), then run the survivor on subject `s`. It is fifteen cells, pure {0, 1, 2}.

This conditional is genuinely lazy.⁷ The discipline this requires is exact: branches must be kept as quoted formulas rather than compiled as combinator sub-terms. Compile a divergent branch and the eager opcode 2 forces it before selection ever happens (the $\mathbb{K} \text{ I } \Omega$ divergence). Thunking the branch keeps it inert until forced; this is similar to stored procedures in conventional Nock using cores with opcode 9.

What the $\text{SKI}\{0, 1, 2\}$ fragment cannot do effectively is manufacture the boolean. IF above routes on a boolean it is handed; to branch on a property of data (atom as zero, cell-ness, etc.), one must first map a datum to $\mathbb{K}$ or $\mathbb{K} \text{ I}$, and that inspection reads an atom's value or tests cell-ness. Neither is expressible in pure $\{0, 1, 2\}$ because these have no notion of reading out structure.⁸ Selecting on a given boolean is free; deciding what the boolean should be requires the cell test (opcode 3), increment, equality (opcode 5), or negation (built with opcode 6 on opcode 5).

This is precisely the shape of Nock's own conditional. Opcode 6 is trivially derivable from opcodes 0–5, and what it adds over the selector above is the data-derived predicate: run a formula b to compute a loobean on the subject, map that loobean to an address (using opcode 4) to pick one of two formulas, and run only the chosen one. The combinatory core we have built here is the selection half; the predicate half is a partial motivation for opcodes 3, 4, and 5 as minimal additions to the SKI basis rather than merely conveniences.

10 Conclusion

The SKI construction in Nock ISA is Turing-complete, but it does not have laziness, normal-form readback, or sharing. Each of these properties marks a boundary where the fragment

⁷Setting the unchosen branch to a crashing formula confirms it never runs: $\text{IF } \text{TRUE } 111 \perp$ returns 111 without crashing, $\text{IF } \text{FALSE } \perp 222$ returns 222, and the control case $\text{IF } \text{TRUE } \perp 222$ does crash. The chosen branch really executes, so the laziness is not an artifact of dead code.

⁸An objection may be made that opcode 0 does have an explicit notion of structure due to cell addressing, but there is no mechanism in non-virtualized (direct) Nock to utilize this without crashing.

is genuinely insufficient and additional tooling beyond $\{0, 1, 2\}$ plus the structural distribution rule is required.

This article makes concrete the claim that Nock opcodes 0, 1, and 2 are sufficient to represent the SKI combinator calculus and are thus Turing-complete (if insufficient to handle nouns natively). The embedding is a constructive proof that the opcode 0, 1, 2 fragment is Turing-complete: the deferred monograph the *Documentary History* expected is rooted in a handful of lines of Nock. The compiler is a homomorphism from SKI application to Nock evaluation, under a fixed call-by-value convention. The combinators are pure construction; opcode 2 appears only in the formula they assemble at application time, carried as quoted data. A second corollary locates the fragment’s edge precisely: a conditional’s *selection* is combinatory and lives in 0, 1, 2, but its *decision*—computing a boolean from data—does not, which motivates Nock’s basis as an SKI core plus a minimal inspection and arithmetic kernel. The Nock ISA has been manifestly Turing-complete since its inception, but this exposition makes a new demonstration explicit and constructive.☒

References

- Curry, Haskell B. and Robert Feys (1958). *Combinatory Logic*. Amsterdam: North-Holland Publishing Company.
- ~hex, James Torre (2024) “combinators.rkt”. URL: <https://github.com/jpt4/nocks/blob/main/combinators.rkt> (visited on ~2026.6.27).
- Kiselyov, Oleg (2018). “ λ to SKI, Semantically: Declarative Pearl.” In: *Functional and Logic Programming*. Ed. by John P. Gallagher and Martin Sulzmann. Cham: Springer International Publishing, pp. 33–50.
- ~lagrev-nocfep, N. E. Davis and Curtis Yarvin ~sorreg-namtyv (2025). “A Documentary History of the Nock Combinator Calculus.” In: *Urbit Systems Technical Journal* 2.1, pp. 155–190. URL: <https://urbitsystems.tech/article/v02-i01/a->

616 documentary-history-of-the-nock-combinator-
617 calculus.
618 Smullyan, Raymond M. (1994). *To Mock a Mockingbird and*
619 *Other Logic Puzzles: : Including an Amazing Adventure in*
620 *Combinatory Logic*. New York: Knopf.